

12. Array Smallest Subarray with sum greater than a given value

```
class Solution {
public:
    int smallestSubWithSum(int x, vector<int>& arr) {
        int n = arr.size();
        int start = 0, sum = 0, minLen = INT_MAX;

        for(int end = 0; end < n; end++) {
            sum += arr[end]; // expand window

            while(sum > x) { // shrink window
                minLen = min(minLen, end - start + 1);
                sum -= arr[start];
                start++;
            }
        }

        return (minLen == INT_MAX) ? 0 : minLen;
    }
};
```

The screenshot displays a coding platform interface with a dark theme. On the left, the 'Output Window' shows 'Compilation Results' for 'Custom Input' by 'Y.O.G.J. (AI Bot)'. It confirms the 'Problem Solved Successfully' with 1112 test cases passed out of 1112, 100% accuracy, and a score of 0.07. Below this, a 'Solve Next' section lists other problems like 'Sorted subsequence of size 3', 'Array Duplicates', and 'Two Sum - Pair with Given Sum'. On the right, the code editor shows the C++ solution for the problem, which is a sliding window algorithm. The code is numbered 1 to 20, and a 'Start Timer' button is visible at the top right of the editor.

```
1 class Solution {
2 public:
3     int smallestSubWithSum(int x, vector<int>& arr) {
4         int n = arr.size();
5         int start = 0, sum = 0, minLen = INT_MAX;
6
7         for(int end = 0; end < n; end++) {
8             sum += arr[end]; // expand window
9
10            while(sum > x) { // shrink window
11                minLen = min(minLen, end - start + 1);
12                sum -= arr[start];
13                start++;
14            }
15        }
16
17        return (minLen == INT_MAX) ? 0 : minLen;
18    }
19 };
20
```